

Error Handling

Generate, catch, and respond to warnings and errors

To make your code more robust, check for edge cases and problematic conditions. The simplest approach is to use an `if` or `switch` statement to check for a specific condition, and then issue an error or warning. For more information, see [Issue Warnings and Errors](#).

Another approach is to catch errors so that the program can continue. For instance, this code creates a variable if loading that variable from a file fails.

```
try
    load(filename,"X")
catch
    warning("Unable to load data. Setting X to an empty array.")
    X = [];
end
```

MATLAB Language Syntax

<code>try</code> , <code>catch</code>	Execute statements and catch resulting errors
---------------------------------------	---

Functions

<code>error</code>	Throw error and display message
<code>warning</code>	Display warning message
<code>lastwarn</code>	Last warning message
<code>assert</code>	Throw error if condition false
<code>onCleanup</code>	Cleanup tasks upon function completion
<code>cancel</code>	Cancel cleanup tasks (<i>Since R2025a</i>)

Topics

Issue Warnings and Errors

To flag unexpected conditions when running a program, issue a warning. To flag fatal problems within the program, throw an error. Unlike warnings, errors halt the execution of a program.

Suppress Warnings

Your program might issue warnings that do not always adversely affect execution. To avoid confusion, you can hide warning messages during execution by changing their states from `'on'` to `'off'`.

Restore Warnings

You can save the warning current states, modify warning states, and restore the original warning states. This technique is useful if you temporarily turn off some warnings and later reinstate the original settings.

Change How Warnings Display

You can control how warnings appear in MATLAB®, including the display of warning suppression information and stack traces.

Use try/catch to Handle Errors

Use a try/catch statement to execute code after your program encounters an error.

Clean Up When Functions Complete

It is a good programming practice to leave your program environment in a clean state that does not interfere with any other program code.

Related Information

Argument Definitions